

Министерство науки и высшего образования Российской Федерации
Федеральное государственное автономное образовательное учреждение
высшего образования

**«Пермский национальный исследовательский политехнический
университет»**

Электротехнический факультет

Выпускающая кафедра: Информационные технологии и
автоматизированные системы (ИТАС)

Направление подготовки: 09.03.04 Программная инженерия

ОТЧЕТ

Лабораторная работа №1

«Бинарные деревья.»

По дисциплине «Основы алгоритмизации и программирования»

Вариант 25

Выполнил: студент группы РИС-25-2Б

Ильинов Фёдор Михайлович

Принял: Доц. Полякова О. А.

Пермь 2026

ПОСТАНОВКА ЗАДАЧИ

Задачи:

Реализовать структуру данных «бинарное дерево поиска» с информационным полем типа double (вариант 25).

Разработать алгоритмы:

Вставки и удаления узлов

Поиска элемента по ключу

Прямого, симметричного и обратного обходов дерева

Балансировки дерева

Вертикальной и горизонтальной печати дерева

Создать графический интерфейс пользователя для взаимодействия с программой.

Реализовать графическую визуализацию структуры дерева.

Найти минимальный элемент в дереве (вариант 25).

АНАЛИЗ.

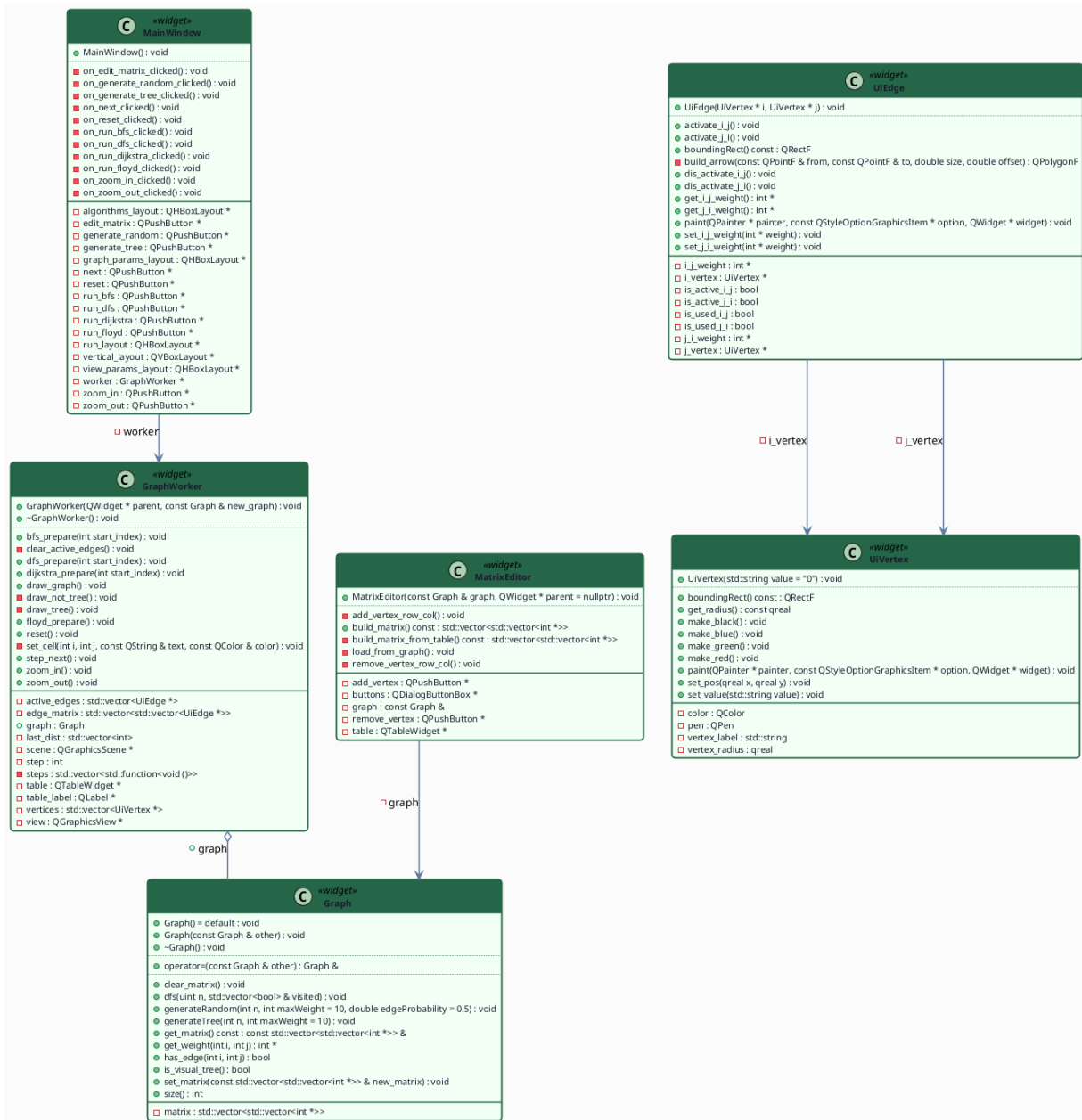
Визуализация бинарного дерева поиска обеспечивается через UiVertex (вершины) и QGraphicsLineItem (рёбра), где вершины наследуют графические элементы Qt, отображают числовой ключ типа double и поддерживают цветовую индикацию (красный — активный, синий — посещённый, зелёный — найденный/минимальный).

Класс BST реализует структуру данных «бинарное дерево поиска» на указателях (левый/правый ребёнок) с методами вставки, удаления, поиска, нахождения минимума, прямого/симметричного/обратного обходов и балансировки через сортировку ключей и рекурсивное построение.

GraphWorker выполняет пошаговую визуализацию операций над деревом, генерирует шаги анимации (подсветка пути при вставке/удалении/поиске/поиске минимума, последовательная подсветка узлов при обходах) и связывает модель BST с графическим представлением.

MainWindow принимает ввод ключа через соответствующее поле сверху, по нажатию кнопок запускает соответствующую операцию через GraphWorker, управляет сбросом анимации (Reset) и пошаговым воспроизведением (Next Step).

UML-ДИАГРАММА



КОД UiVertex.h

```
//
// Created by localuser on 5/12/26.
//
```

```
#ifndef QTGRAPHVISUALIZATION_UIVERTEX_H
#define QTGRAPHVISUALIZATION_UIVERTEX_H
#include <QGraphicsScene>
```

```

#include <QGraphicsItem>
#include <QPainter>

class UiVertex : public QGraphicsItem {
    std::string vertex_label = "0";
    qreal vertex_radius = 30;

    QColor color = Qt::black;
    QPen pen = QPen(Qt::black, 3);
public:
    UiVertex(std::string value = "0");

    void set_pos(qreal x, qreal y);
    void set_value(std::string value);
    const qreal get_radius();

    void make_red();
    void make_green();
    void make_blue();
    void make_black();
    QRectF boundingRect() const override;

    void paint(QPainter *painter,
               const QStyleOptionGraphicsItem *option,
               QWidget *widget) override;
};

#endif //QTGRAPHVISUALIZATION_UIVERTEX_H

```

КОД UiVertex.cpp

```
//  
// Created by localuser on 5/12/26.  
//  
  
#include "UiVertex.h"  
  
UiVertex::UiVertex(std::string value) {  
    vertex_label = value;  
}  
  
QRectF UiVertex::boundingRect() const {  
    return QRectF(-vertex_radius, -vertex_radius,  
        vertex_radius * 2,  
        vertex_radius * 2);  
}  
  
void UiVertex::paint(QPainter *painter,  
    const QStyleOptionGraphicsItem *,  
    QWidget *) {  
  
    painter->setPen(pen);  
  
    painter->setBrush(Qt::white);  
    painter->drawEllipse(boundingRect());  
  
    painter->setPen(Qt::black);  
    painter->drawText(boundingRect(),  
        Qt::AlignCenter,  
        QString::fromStdString(vertex_label));  
}  
  
void UiVertex::set_pos(qreal x, qreal y) {  
    setPos(x, y);  
}  
  
void UiVertex::set_value(std::string value) {  
    vertex_label = value;  
    update();  
}  
  
const qreal UiVertex::get_radius() {  
    return vertex_radius;  
}
```

```
void UiVertex::make_red() {  
    pen = QPen(Qt::red, 5);  
    update();  
}  
void UiVertex::make_blue() {  
    pen = QPen(Qt::blue, 5);  
    update();  
}  
void UiVertex::make_green() {  
    pen = QPen(Qt::green, 3);  
    update();  
}  
  
void UiVertex::make_black() {  
    pen = QPen(Qt::black, 3);  
    update();  
}
```

КОД MainWindow.h

```
#ifndef QTGRAPHVISUALIZATION_MAINWINDOW_H
#define QTGRAPHVISUALIZATION_MAINWINDOW_H
```

```
#include <QWidget>
#include <QVBoxLayout>
#include <QHBoxLayout>
#include <QPushButton>
#include <QDoubleSpinBox>
#include <QLabel>
#include "GraphWorker.h"
```

```
class MainWindow : public QWidget {
    Q_OBJECT
```

```
    QDoubleSpinBox *key_input;
    QPushButton *btn_insert;
    QPushButton *btn_remove;
    QPushButton *btn_search;
    QPushButton *btn_find_min;
    QPushButton *btn_preorder;
    QPushButton *btn_inorder;
    QPushButton *btn_postorder;
    QPushButton *btn_balance;
    QPushButton *btn_reset;
    QPushButton *btn_next;
    QPushButton *btn_zoom_in;
    QPushButton *btn_zoom_out;
```

```
    GraphWorker *worker;
```

```
public:
    MainWindow();
```

```
private slots:
    void on_insert_clicked();
    void on_remove_clicked();
    void on_search_clicked();
    void on_find_min_clicked();
    void on_preorder_clicked();
    void on_inorder_clicked();
    void on_postorder_clicked();
    void on_balance_clicked();
    void on_reset_clicked();
```



```
void on_next_clicked();  
void on_zoom_in_clicked();  
void on_zoom_out_clicked();  
};
```

```
#endif
```

КОД MainWindow.cpp

```
#include "MainWindow.h"
```

```
MainWindow::MainWindow() : QWidget(nullptr) {  
    auto *vertical_layout = new QVBoxLayout(this);
```

```
    worker = new GraphWorker(this);  
    worker->draw();
```

```
    auto *input_layout = new QHBoxLayout;  
    key_input = new QDoubleSpinBox(this);  
    key_input->setRange(-10000.0, 10000.0);  
    key_input->setDecimals(2);  
    key_input->setSingleStep(1.0);
```

```
    btn_insert = new QPushButton("Insert", this);  
    btn_remove = new QPushButton("Remove", this);  
    btn_search = new QPushButton("Search", this);  
    btn_find_min = new QPushButton("Find Min", this);
```

```
    input_layout->addWidget(new QLabel("Key:", this));  
    input_layout->addWidget(key_input);  
    input_layout->addWidget(btn_insert);  
    input_layout->addWidget(btn_remove);  
    input_layout->addWidget(btn_search);  
    input_layout->addWidget(btn_find_min);  
    vertical_layout->addLayout(input_layout);
```

```
    vertical_layout->addWidget(worker);
```

```
    auto *traversal_layout = new QHBoxLayout;  
    btn_preorder = new QPushButton("PreOrder", this);  
    btn_inorder = new QPushButton("InOrder", this);  
    btn_postorder = new QPushButton("PostOrder", this);  
    btn_balance = new QPushButton("Balance", this);
```

```
    traversal_layout->addWidget(btn_preorder);  
    traversal_layout->addWidget(btn_inorder);
```

```

traversal_layout->addWidget(btn_postorder);
traversal_layout->addWidget(btn_balance);
vertical_layout->addLayout(traversal_layout);

auto *view_layout = new QHBoxLayout;
btn_zoom_in = new QPushButton("Zoom In", this);
btn_zoom_out = new QPushButton("Zoom Out", this);
btn_reset = new QPushButton("Reset", this);
btn_next = new QPushButton("Next Step", this);

view_layout->addWidget(btn_zoom_in);
view_layout->addWidget(btn_zoom_out);
view_layout->addWidget(btn_reset);
view_layout->addWidget(btn_next);
vertical_layout->addLayout(view_layout);

connect(btn_insert, &QPushButton::clicked, this, &MainWindow::on_insert_clicked);
connect(btn_remove, &QPushButton::clicked, this, &MainWindow::on_remove_clicked);
connect(btn_search, &QPushButton::clicked, this, &MainWindow::on_search_clicked);
connect(btn_find_min, &QPushButton::clicked, this, &MainWindow::on_find_min_clicked);
connect(btn_preorder, &QPushButton::clicked, this, &MainWindow::on_preorder_clicked);
connect(btn_inorder, &QPushButton::clicked, this, &MainWindow::on_inorder_clicked);
connect(btn_postorder, &QPushButton::clicked, this, &MainWindow::on_postorder_clicked);
connect(btn_balance, &QPushButton::clicked, this, &MainWindow::on_balance_clicked);
connect(btn_reset, &QPushButton::clicked, this, &MainWindow::on_reset_clicked);
connect(btn_next, &QPushButton::clicked, this, &MainWindow::on_next_clicked);
connect(btn_zoom_in, &QPushButton::clicked, this, &MainWindow::on_zoom_in_clicked);
connect(btn_zoom_out, &QPushButton::clicked, this, &MainWindow::on_zoom_out_clicked);
}

void MainWindow::on_insert_clicked() {
    worker->reset();
    worker->insert_prepare(key_input->value());
}

void MainWindow::on_remove_clicked() {
    worker->reset();
    worker->remove_prepare(key_input->value());
}

void MainWindow::on_search_clicked() {
    worker->reset();
    worker->search_prepare(key_input->value());
}

```

```
void MainWindow::on_find_min_clicked() {
    worker->reset();
    worker->find_min_prepare();
}

void MainWindow::on_preorder_clicked() {
    worker->reset();
    worker->traversal_prepare(worker->tree.preOrder());
}

void MainWindow::on_inorder_clicked() {
    worker->reset();
    worker->traversal_prepare(worker->tree.inOrder());
}

void MainWindow::on_postorder_clicked() {
    worker->reset();
    worker->traversal_prepare(worker->tree.postOrder());
}

void MainWindow::on_balance_clicked() {
    worker->reset();
    worker->balance_tree();
}

void MainWindow::on_reset_clicked() {
    worker->reset();
}

void MainWindow::on_next_clicked() {
    worker->step_next();
}

void MainWindow::on_zoom_in_clicked() {
    worker->zoom_in();
}

void MainWindow::on_zoom_out_clicked() {
    worker->zoom_out();
}
```

КОД GraphWorker.h

```
#ifndef QTGRAPHVISUALIZATION_GRAPHWORKER_H
#define QTGRAPHVISUALIZATION_GRAPHWORKER_H

#include <QGraphicsView>
#include <QWidget>
#include <vector>
#include <unordered_map>
#include <functional>
#include "UiVertex.h"
#include "BST.h"

class GraphWorker : public QWidget {
    Q_OBJECT

    std::vector<UiVertex *> vertices;
    std::unordered_map<Node *, UiVertex *> node_vertex_map;
    std::vector<std::function<void()>> steps;
    int step = 0;

    QGraphicsScene *scene;
    QGraphicsView *view;

    void compute_positions(Node *node, int depth, int &leaf_counter,
                          std::unordered_map<Node *, double> &x_pos,
                          std::unordered_map<Node *, double> &y_pos);
    void draw_tree();
    static std::string key_str(double key);

public:
    explicit GraphWorker(QWidget *parent);
    ~GraphWorker() override;

    BST tree;
    void draw();
    void zoom_in();
    void zoom_out();

    void insert_prepare(double key);
    void remove_prepare(double key);
    void search_prepare(double key);
    void find_min_prepare();
    void traversal_prepare(const std::vector<double> &order);
    void balance_tree();
};
```

```

    void step_next();
    void reset();
};

```

```

#endif

```

КОД GraphWorker.cpp

```

#include "GraphWorker.h"

```

```

#include <QVBoxLayout>
#include <QGraphicsLineItem>
#include <QString>
#include <algorithm>

```

```

GraphWorker::GraphWorker(QWidget *parent) : QWidget(parent) {
    scene = new QGraphicsScene(this);
    view = new QGraphicsView(scene, this);

    view->setRenderHint(QPainter::Antialiasing);
    view->setSizePolicy(QSizePolicy::Expanding, QSizePolicy::Expanding);
    view->setTransformationAnchor(QGraphicsView::AnchorUnderMouse);
    view->setInteractive(true);
    view->setDragMode(QGraphicsView::ScrollHandDrag);

    auto *layout = new QVBoxLayout(this);
    layout->setContentsMargins(0, 0, 0, 0);
    layout->addWidget(view);
    setLayout(layout);

    setSizePolicy(QSizePolicy::Expanding, QSizePolicy::Expanding);
}

```

```

GraphWorker::~GraphWorker() {
    for (auto *v : vertices) delete v;
}

```

```

std::string GraphWorker::key_str(double key) {
    QString s = QString::number(key, 'f', 2);
    while (s.contains('.') && (s.endsWith('0') || s.endsWith('.')))
        s.chop(1);
    return s.toStdString();
}

```

```

void GraphWorker::compute_positions(Node *node, int depth, int &leaf_counter,

```

```

        std::unordered_map<Node *, double> &x_pos,
        std::unordered_map<Node *, double> &y_pos) {
const double H_STEP = 200.0;
const double V_STEP = 200.0;

y_pos[node] = depth * V_STEP + 60.0;

if (!node->left && !node->right) {
    x_pos[node] = leaf_counter * H_STEP + 60.0;
    leaf_counter++;
    return;
}

if (node->left)
    compute_positions(node->left, depth + 1, leaf_counter, x_pos, y_pos);
if (node->right)
    compute_positions(node->right, depth + 1, leaf_counter, x_pos, y_pos);

double left_x = node->left ? x_pos[node->left] : (leaf_counter * H_STEP + 60.0);
double right_x = node->right ? x_pos[node->right] : (leaf_counter * H_STEP + 60.0);
x_pos[node] = (left_x + right_x) / 2.0;
}

void GraphWorker::draw_tree() {
    scene->clear();
    vertices.clear();
    node_vertex_map.clear();

    if (!tree.getRoot()) return;

    std::unordered_map<Node *, double> x_pos, y_pos;
    int leaf_counter = 0;
    compute_positions(tree.getRoot(), 0, leaf_counter, x_pos, y_pos);

    double min_x = 1e9, max_x = -1e9, min_y = 1e9, max_y = -1e9;
    for (auto &[node, x] : x_pos) {
        min_x = std::min(min_x, x);
        max_x = std::max(max_x, x);
        min_y = std::min(min_y, y_pos[node]);
        max_y = std::max(max_y, y_pos[node]);
    }

    QPointF center = view->mapToScene(view->viewport()->rect().center());
    double shift_x = center.x() - (min_x + max_x) / 2.0;
    double shift_y = center.y() - (min_y + max_y) / 2.0;

```

```

std::function<void(Node *)> draw_edges = [&](Node *node) {
    if (!node) return;
    if (node->left) {
        auto *line = scene->addLine(
            x_pos[node] + shift_x, y_pos[node] + shift_y,
            x_pos[node->left] + shift_x, y_pos[node->left] + shift_y,
            QPen(Qt::black, 3));
        line->setZValue(-1);
    }
    if (node->right) {
        auto *line = scene->addLine(
            x_pos[node] + shift_x, y_pos[node] + shift_y,
            x_pos[node->right] + shift_x, y_pos[node->right] + shift_y,
            QPen(Qt::black, 3));
        line->setZValue(-1);
    }
    draw_edges(node->left);
    draw_edges(node->right);
};
draw_edges(tree.getRoot());

std::function<void(Node *)> draw_vertices = [&](Node *node) {
    if (!node) return;
    auto *v = new UiVertex(key_str(node->key));
    v->set_pos(x_pos[node] + shift_x, y_pos[node] + shift_y);
    scene->addItem(v);
    vertices.push_back(v);
    node_vertex_map[node] = v;
    draw_vertices(node->left);
    draw_vertices(node->right);
};
draw_vertices(tree.getRoot());
}

void GraphWorker::draw() {
    draw_tree();
}

void GraphWorker::zoom_in() {
    view->setTransformationAnchor(QGraphicsView::AnchorViewCenter);
    view->scale(1.2, 1.2);
}

void GraphWorker::zoom_out() {

```

```

view->setTransformationAnchor(QGraphicsView::AnchorViewCenter);
view->scale(1.0 / 1.2, 1.0 / 1.2);
}

```

```

void GraphWorker::insert_prepare(double key) {
    steps.clear();
    step = 0;

    if (!tree.getRoot()) {
        tree.insert(key);
        draw();
        return;
    }
}

```

```

std::vector<Node *> path;
Node *current = tree.getRoot();
while (current) {
    path.push_back(current);
    if (key < current->key) {
        if (!current->left) break;
        current = current->left;
    } else if (key > current->key) {
        if (!current->right) break;
        current = current->right;
    } else {
        break;
    }
}
}

```

```

for (size_t i = 0; i < path.size(); i++) {
    Node *node = path[i];
    if (i == 0) {
        steps.push_back([this, node]() {
            if (node_vertex_map.count(node))
                node_vertex_map[node]->make_red();
        });
    } else {
        Node *prev = path[i - 1];
        steps.push_back([this, prev, node]() {
            if (prev && node_vertex_map.count(prev))
                node_vertex_map[prev]->make_blue();
            if (node && node_vertex_map.count(node))
                node_vertex_map[node]->make_red();
        });
    }
}
}

```



```

    }

    steps.push_back([this, key]() {
        tree.insert(key);
        draw();
    });
}

void GraphWorker::remove_prepare(double key) {
    steps.clear();
    step = 0;

    Node *current = tree.getRoot();
    if (!current) return;

    Node *prev = nullptr;
    while (current) {
        steps.push_back([this, current, prev]() {
            if (prev && node_vertex_map.count(prev))
                node_vertex_map[prev]->make_blue();
            if (current && node_vertex_map.count(current))
                node_vertex_map[current]->make_red();
        });
        prev = current;
        if (key < current->key)
            current = current->left;
        else if (key > current->key)
            current = current->right;
        else
            break;
    }

    steps.push_back([this, key]() {
        tree.remove(key);
        draw();
    });
}

void GraphWorker::search_prepare(double key) {
    steps.clear();
    step = 0;

    Node *current = tree.getRoot();
    if (!current) return;

```

```

Node *prev = nullptr;
while (current) {
    steps.push_back([this, current, prev]() {
        if (prev && node_vertex_map.count(prev))
            node_vertex_map[prev]->make_blue();
        if (current && node_vertex_map.count(current))
            node_vertex_map[current]->make_red();
    });
    prev = current;
    if (key < current->key)
        current = current->left;
    else if (key > current->key)
        current = current->right;
    else
        break;
}

if (current && current->key == key) {
    steps.push_back([this, current]() {
        if (current && node_vertex_map.count(current))
            node_vertex_map[current]->make_green();
    });
} else {
    steps.push_back([this, prev]() {
        if (prev && node_vertex_map.count(prev))
            node_vertex_map[prev]->make_green();
    });
}
}

void GraphWorker::find_min_prepare() {
    steps.clear();
    step = 0;

    Node *current = tree.getRoot();
    if (!current) return;

    Node *prev = nullptr;
    while (current) {
        steps.push_back([this, current, prev]() {
            if (prev && node_vertex_map.count(prev))
                node_vertex_map[prev]->make_blue();
            if (current && node_vertex_map.count(current))
                node_vertex_map[current]->make_red();
        });
    }
}

```

```

    prev = current;
    current = current->left;
}

steps.push_back([this, min_node = prev]() {
    if (min_node && node_vertex_map.count(min_node))
        node_vertex_map[min_node]->make_green();
});
}

void GraphWorker::traversal_prepare(const std::vector<double> &order) {
    steps.clear();
    step = 0;
    if (order.empty()) return;

    for (auto &[node, vertex] : node_vertex_map)
        vertex->make_black();

    steps.push_back([this, key = order[0]]() {
        Node *node = tree.search(key);
        if (node && node_vertex_map.count(node))
            node_vertex_map[node]->make_red();
    });

    for (size_t i = 1; i < order.size(); i++) {
        double prev_key = order[i - 1];
        double curr_key = order[i];
        steps.push_back([this, prev_key, curr_key]() {
            Node *prev_node = tree.search(prev_key);
            Node *curr_node = tree.search(curr_key);
            if (prev_node && node_vertex_map.count(prev_node))
                node_vertex_map[prev_node]->make_blue();
            if (curr_node && node_vertex_map.count(curr_node))
                node_vertex_map[curr_node]->make_red();
        });
    }

    steps.push_back([this, key = order.back]() {
        Node *node = tree.search(key);
        if (node && node_vertex_map.count(node))
            node_vertex_map[node]->make_green();
    });
}

void GraphWorker::balance_tree() {

```

```

    tree.balance();
    draw();
}

void GraphWorker::step_next() {
    if (step >= static_cast<int>(steps.size())) return;
    steps[step]();
    step++;
}

void GraphWorker::reset() {
    steps.clear();
    step = 0;
    for (auto &[node, vertex] : node_vertex_map)
        vertex->make_black();
}

```

КОД BST.h

```

#ifdef QTGRAPHVISUALIZATION_BST_H
#define QTGRAPHVISUALIZATION_BST_H

#include <vector>

struct Node {
    double key;
    Node *left = nullptr;
    Node *right = nullptr;
    explicit Node(double k) : key(k) {}
};

class BST {
    Node *root = nullptr;

    Node *insertRec(Node *node, double key);
    Node *removeRec(Node *node, double key);
    Node *findMinNode(Node *node) const;
    Node *searchRec(Node *node, double key) const;
    void inorderRec(Node *node, std::vector<double> &result) const;
    void preorderRec(Node *node, std::vector<double> &result) const;
    void postorderRec(Node *node, std::vector<double> &result) const;
    Node *balanceRec(const std::vector<double> &keys, int l, int r);
    void clearRec(Node *node);

public:
    BST() = default;

```

```

~BST();

void insert(double key);
void remove(double key);
Node *search(double key) const;
Node *findMin() const;
std::vector<double> inOrder() const;
std::vector<double> preOrder() const;
std::vector<double> postOrder() const;
void balance();
void clear();
Node *getRoot() const { return root; }
};

#endif

```

КОД BST.cpp

```

#include "BST.h"

BST::~BST() {
    clear();
}

void BST::clear() {
    clearRec(root);
    root = nullptr;
}

void BST::clearRec(Node *node) {
    if (!node) return;
    clearRec(node->left);
    clearRec(node->right);
    delete node;
}

void BST::insert(double key) {
    root = insertRec(root, key);
}

Node *BST::insertRec(Node *node, double key) {
    if (!node) return new Node(key);
    if (key < node->key)
        node->left = insertRec(node->left, key);
    else if (key > node->key)
        node->right = insertRec(node->right, key);
}

```

```

    return node;
}

void BST::remove(double key) {
    root = removeRec(root, key);
}

Node *BST::removeRec(Node *node, double key) {
    if (!node) return nullptr;
    if (key < node->key)
        node->left = removeRec(node->left, key);
    else if (key > node->key)
        node->right = removeRec(node->right, key);
    else {
        if (!node->left) {
            Node *temp = node->right;
            delete node;
            return temp;
        }
        if (!node->right) {
            Node *temp = node->left;
            delete node;
            return temp;
        }
        Node *temp = findMinNode(node->right);
        node->key = temp->key;
        node->right = removeRec(node->right, temp->key);
    }
    return node;
}

Node *BST::findMinNode(Node *node) const {
    while (node && node->left)
        node = node->left;
    return node;
}

Node *BST::search(double key) const {
    return searchRec(root, key);
}

Node *BST::searchRec(Node *node, double key) const {
    if (!node || node->key == key) return node;
    if (key < node->key)
        return searchRec(node->left, key);

```

```

    return searchRec(node->right, key);
}

Node *BST::findMin() const {
    return findMinNode(root);
}

std::vector<double> BST::inOrder() const {
    std::vector<double> result;
    inorderRec(root, result);
    return result;
}

void BST::inorderRec(Node *node, std::vector<double> &result) const {
    if (!node) return;
    inorderRec(node->left, result);
    result.push_back(node->key);
    inorderRec(node->right, result);
}

std::vector<double> BST::preOrder() const {
    std::vector<double> result;
    preorderRec(root, result);
    return result;
}

void BST::preorderRec(Node *node, std::vector<double> &result) const {
    if (!node) return;
    result.push_back(node->key);
    preorderRec(node->left, result);
    preorderRec(node->right, result);
}

std::vector<double> BST::postOrder() const {
    std::vector<double> result;
    postorderRec(root, result);
    return result;
}

void BST::postorderRec(Node *node, std::vector<double> &result) const {
    if (!node) return;
    postorderRec(node->left, result);
    postorderRec(node->right, result);
    result.push_back(node->key);
}

```

```

void BST::balance() {
    std::vector<double> keys = inOrder();
    clear();
    root = balanceRec(keys, 0, static_cast<int>(keys.size() - 1));
}

Node *BST::balanceRec(const std::vector<double> &keys, int l, int r) {
    if (l > r) return nullptr;
    int mid = (l + r) / 2;
    auto *node = new Node(keys[mid]);
    node->left = balanceRec(keys, l, mid - 1);
    node->right = balanceRec(keys, mid + 1, r);
    return node;
}

```

КОД main.cpp

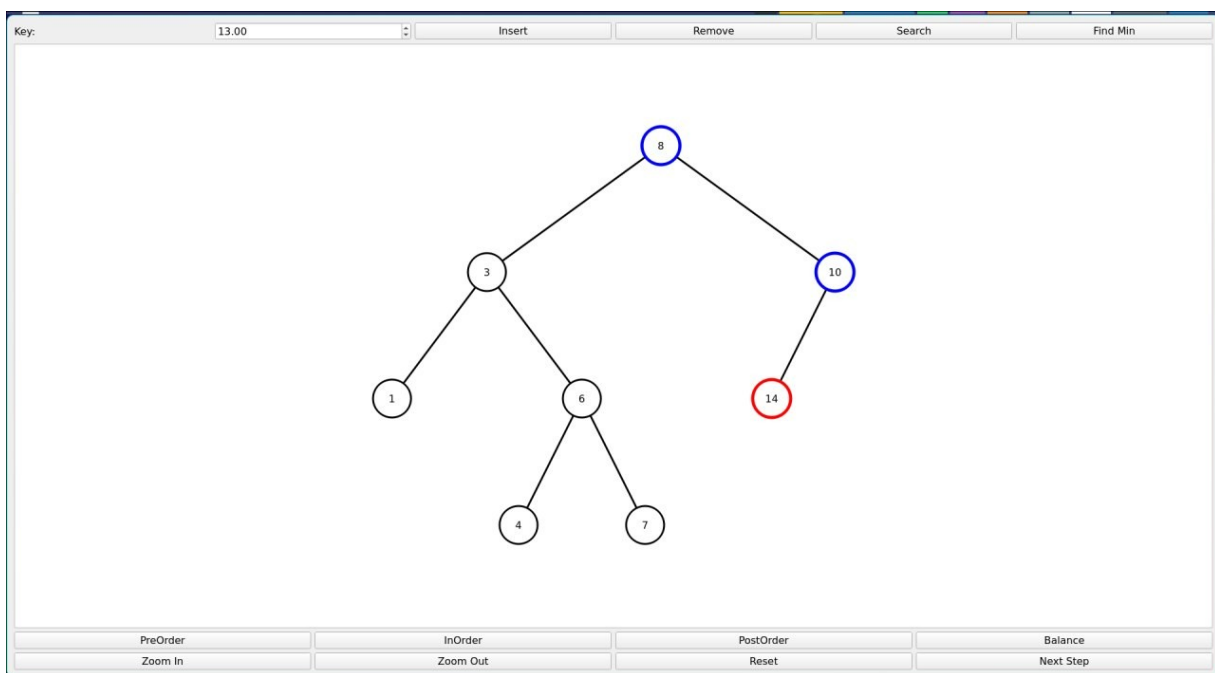
```

#include <QApplication>
#include <QPushButton>
#include "MainWindow.h"

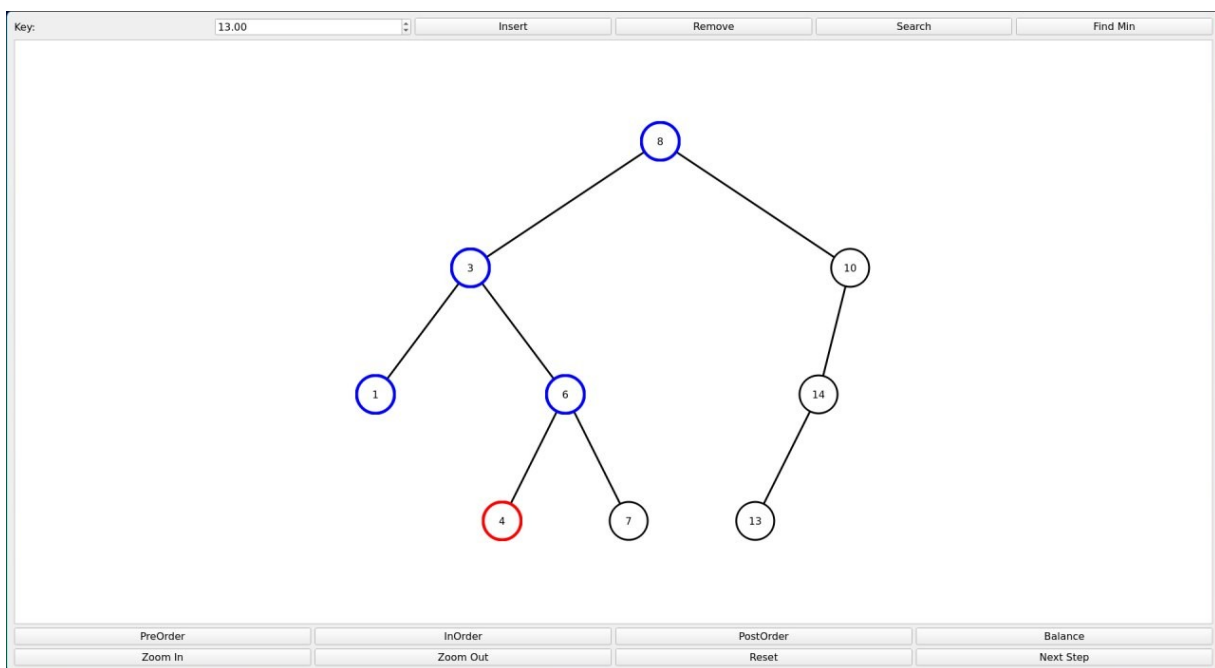
int main(int argc, char* argv[]) {
    QApplication a(argc, argv);
    MainWindow window;
    window.show();
    return QApplication::exec();
}

```

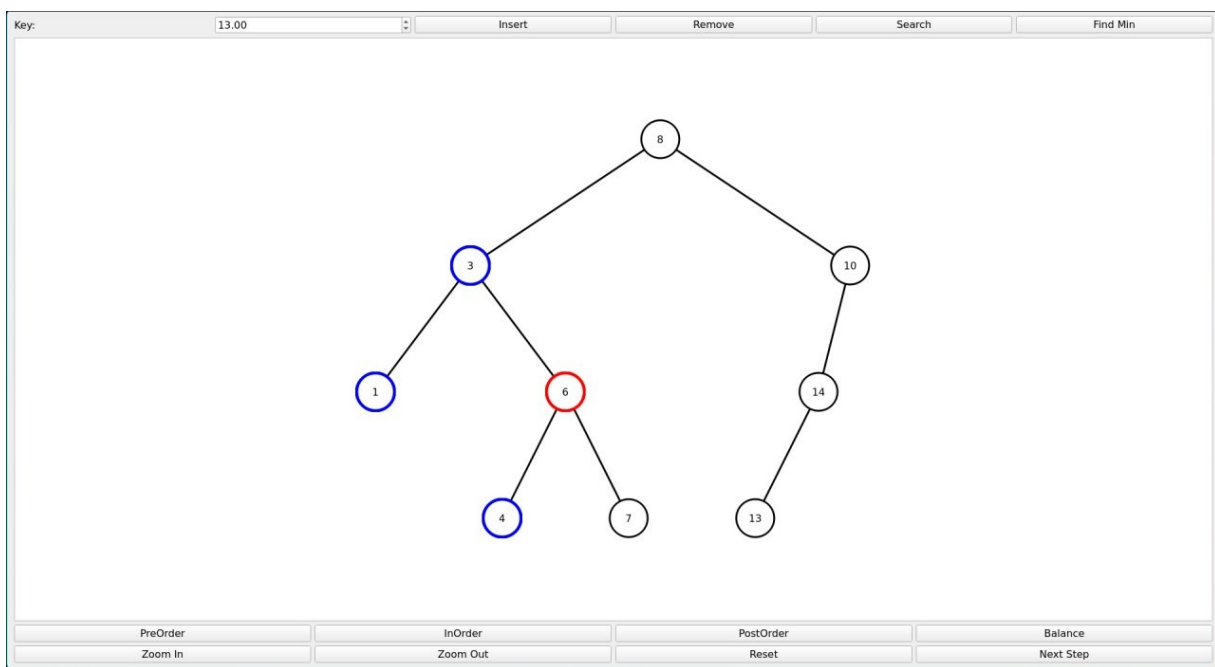
РЕЗУЛЬТАТ ВЫПОЛНЕНИЯ



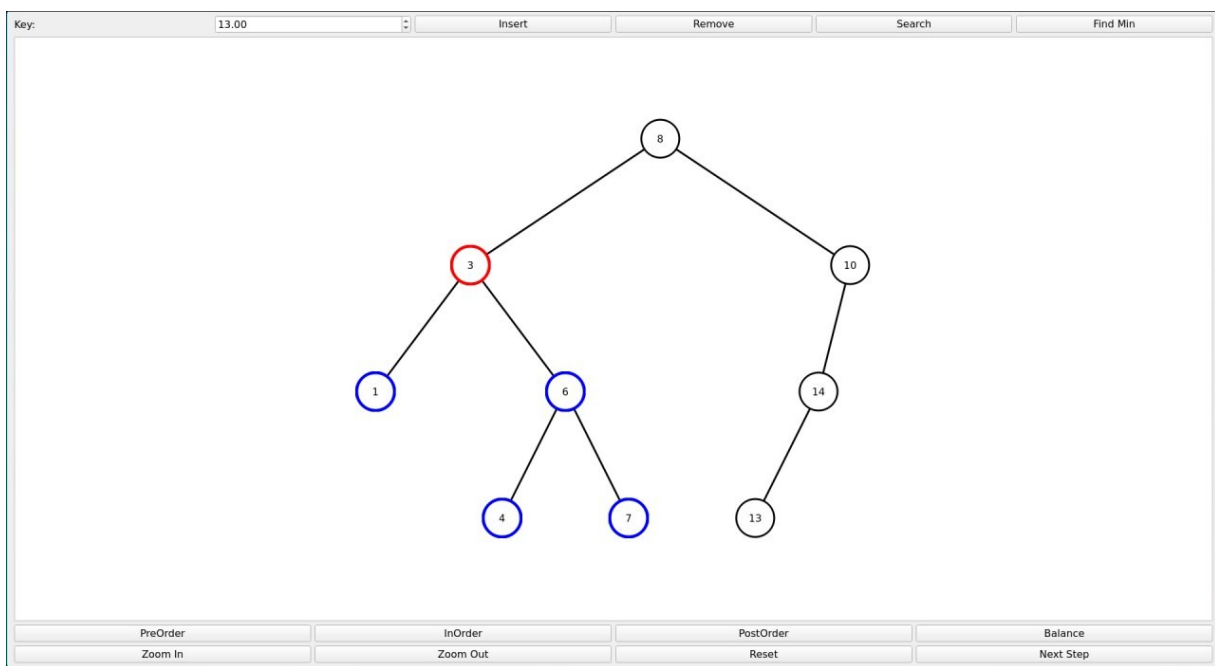
Последний шаг добавления вершины со значением 13



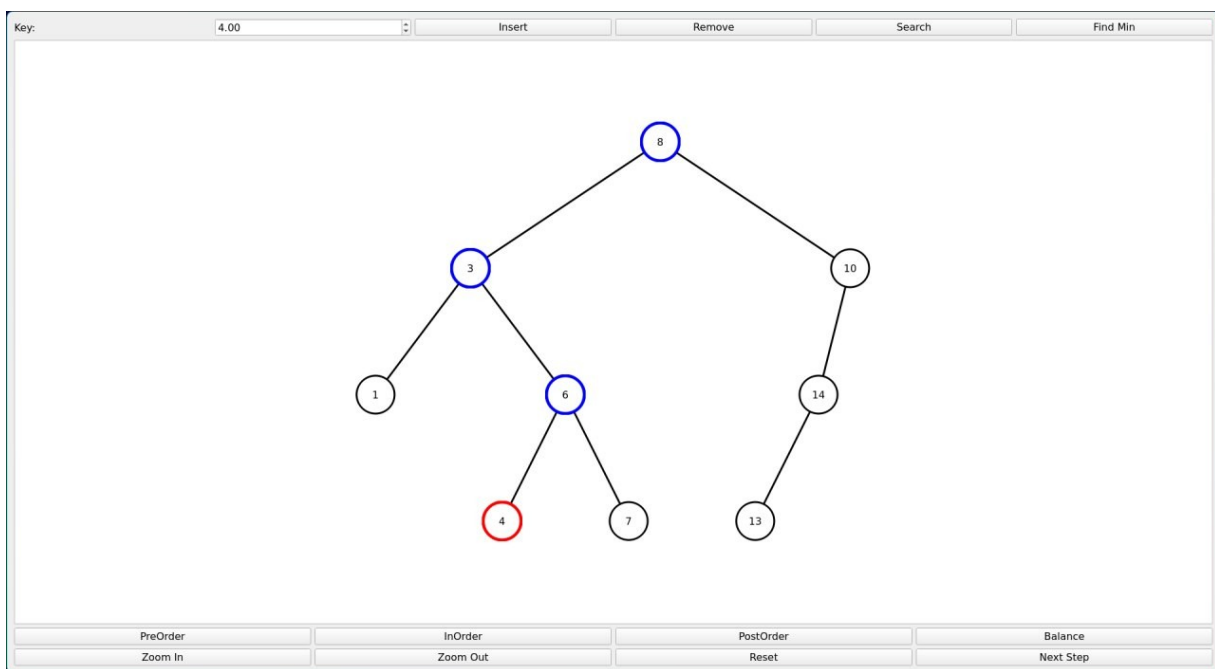
PreOrder: 8→3→1→3→6→4



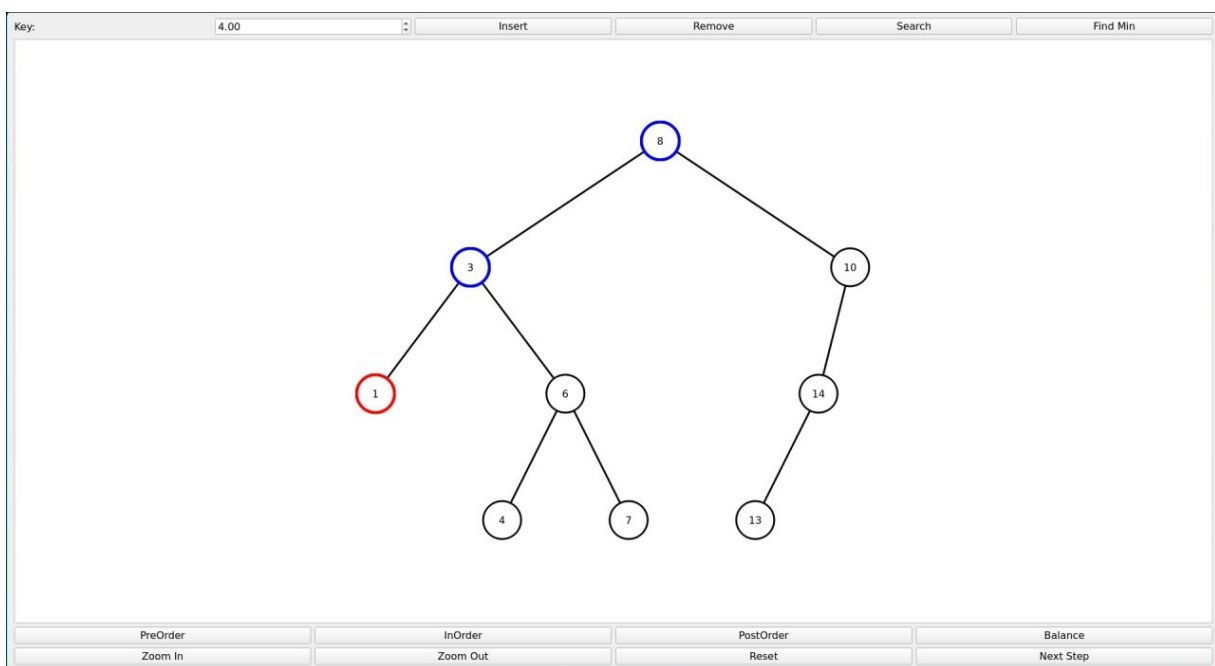
InOrder: 1→3→4→6



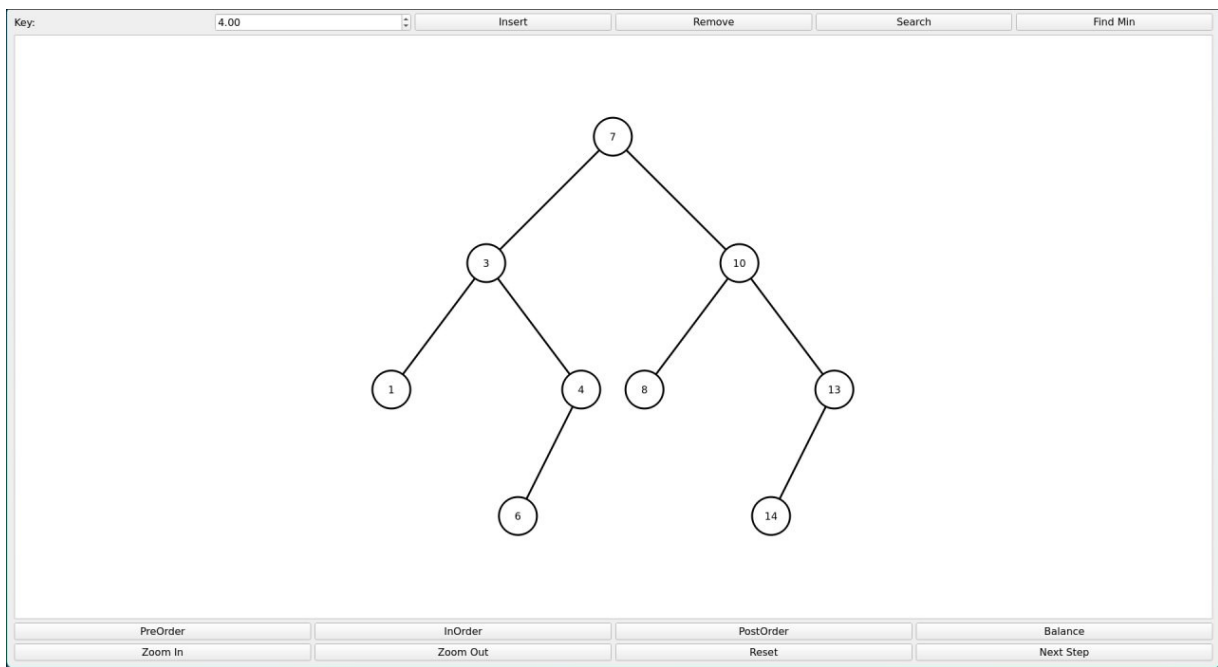
PostOrder: 1→4→7→6→3



Последний шаг поиска вершины со значением 4



Последний шаг поиска минимального значения



Сбалансированное дерево